- *LangChain.js*—LangChain.js is used for prompt management, document parsing, and vectorization. We use the community-supported `UpstashVectorStore` package to interface with the vector database, enabling semantic search through retrievers that match user queries with relevant document chunks.
- *Upstash Redis*—For external data storage, we employ Upstash Redis as a simple yet performant solution. This choice meets our intermediate data needs while allowing for easy scalability to a relational database like PostgreSQL if required.
- *Upstash Vector*—Upstash Vector serves as our dedicated vector database, optimized for storing and searching vectorized document embeddings. We create a separate client with secure access credentials to interact with this service for enabling efficient similarity search operations suitable for RAG workflows. We utilize a Vector index with 768 output dimensions to match the Google AI embedding model requirements.
- *Clerk.js*—Authentication and user management are handled by Clerk.js, which offloads the complexity and security risks of managing authentication flows.

All the technologies used in this project have been explored in previous chapters, except for the Upstash Vector store. But thanks to LangChain.js's modular retriever interface, integrating Upstash Vector is straightforward. LangChain treats Upstash Vector like any other retrieval service, so it allows seamless management of vector embeddings and semantic search capabilities.

Application architecture

The application follows a modular architecture. The frontend layer consists of React components with Tailwind CSS for a responsive UI. It uses the shadcn component library for the common components like buttons, alerts, and textboxes. Now let's explore in detail the individual features and how they are implemented.

DocumentUploader

The `DocumentUploader` component provides an intuitive drag-and-drop interface for users to upload documents into a selected knowledge base. It supports multiple file types, including PDF and DOCX. Users can also browse their local files, preview selected documents, remove any before uploading, and receive status updates during the upload process.

Knowledge base upload page

The knowledge base upload page integrates the `DocumentUploader` within the context of a specific knowledge base. It allows users to manage document uploads tied to that knowledge base and view an overview of uploaded documents. This page acts as the gateway for preparing knowledge bases for conversational AI interactions.

API routes

The application backend is structured using Next.js API routes, organized under the `/api` directory to handle various operations: